

Application datasetManager

Gestion d'un catalogue de datasets

TP PYTHON • FORMATION INTELLIGENCE ARTIFICIELLE

Réalisé par **Khadija Ngom**

Dakar — 2026

Sommaire

Sommaire	2
Contexte	3
Partie 1 — Types de base, variables, entrées et sorties	3
Question 3 — Saisie des métadonnées d'un dataset	3
Question 4 — Affichage d'un résumé formaté	3
Résultat de l'exécution	3
Partie 2 — Structures de contrôle	4
Question 5 — Créer un menu interactif	4
Résultat de l'exécution	5
Partie 3 — Dictionnaires	5
Question 6 — Créer un dictionnaire pour stocker les métadonnées	5
Résultat de l'exécution	6
Partie 4 — Tuples	6
Question 7 — Créer un tuple des domaines autorisés	6
Question 8 — Vérifier que le domaine saisi appartient au tuple	7
Résultat de l'exécution	7
Partie 5 — Listes	7
Question 9 — Créer la liste des datasets	8
Question 10 — Ajouter, afficher, rechercher, trier, modifier, supprimer	8
Résultat de l'exécution	9

Astuce : clic droit sur le sommaire → « Mettre à jour les champs » pour l'actualiser.

Contexte

Dans une entreprise spécialisée en Intelligence Artificielle, les Data Scientists manipulent quotidiennement des centaines de datasets aux formats CSV et JSON. Avant les traitements avec Pandas, ils souhaitent une application pour gérer ce catalogue.

Ce TP consiste à développer progressivement une application console en Python permettant de : gérer un catalogue de datasets, enregistrer leurs caractéristiques, effectuer des recherches, afficher des statistiques, sauvegarder et recharger les données depuis des fichiers, et gérer les erreurs d'utilisation.

Partie 1 — Types de base, variables, entrées et sorties

Cette partie met en place la saisie des métadonnées d'un dataset, puis l'affichage d'un résumé formaté.

NOTIONS ABORDEES types de base (str, int, float, bool) · variables · fonction d'entrée input() · fonction de sortie print() · conversion de type · chaînes formatées (f-strings).

Question 3 — Saisie des métadonnées d'un dataset

On demande le nom, le domaine, le nombre de lignes et de colonnes, la taille en Mo, le format et le caractère public. input() renvoyant toujours du texte, les valeurs numériques sont converties avec int() et float(), et le booléen à partir d'une comparaison.

REPONSE

```
# --- Saisie des métadonnées du dataset ---
nom = input("Nom du dataset : ")
domaine = input("Domaine : ")
lignes = int(input("Nombre de lignes : "))
colonnes = int(input("Nombre de colonnes : "))
taille = float(input("Taille en Mo : "))
format_dataset = input("Format (csv ou json) : ")
public = input("Public ? (true ou false) : ") == "true"
```

Question 4 — Affichage d'un résumé formaté

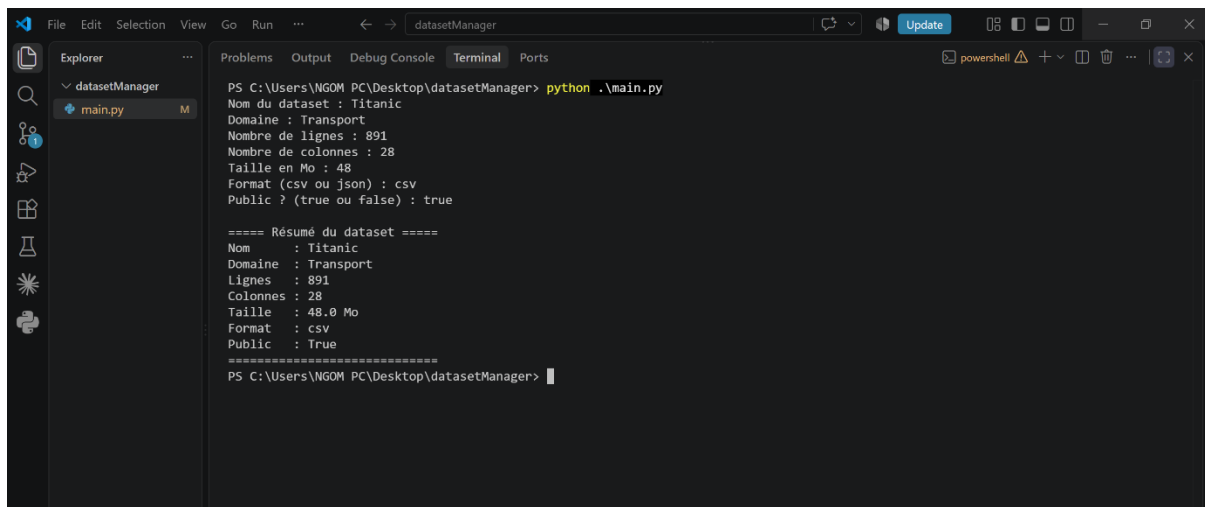
Les informations saisies sont réaffichées dans un résumé encadré à l'aide de f-strings, qui insèrent la valeur de chaque variable directement dans le texte.

REPONSE

```
# --- Affichage du résumé ---
print("\n==== Résumé du dataset =====")
print(f"Nom      : {nom}")
print(f"Domaine   : {domaine}")
print(f"Lignes    : {lignes}")
print(f"Colonnes   : {colonnes}")
print(f"Taille     : {taille} Mo")
print(f"Format     : {format_dataset}")
print(f"Public     : {public}")
print("=====")
```

Résultat de l'exécution

Exécution du programme via la commande python main.py, avec un jeu de valeurs d'exemple :



```
PS C:\Users\NGOM PC\Desktop\datasetManager> python .\main.py
Nom du dataset : Titanic
Domaine : Transport
Nombre de lignes : 891
Nombre de colonnes : 28
Taille en Mo : 48
Format (csv ou json) : csv
Public ? (true ou false) : true

==== Résumé du dataset ====
Nom      : Titanic
Domaine  : Transport
Lignes   : 891
Colonnes : 28
Taille   : 48.0 Mo
Format   : csv
Public   : True
=====
PS C:\Users\NGOM PC\Desktop\datasetManager>
```

La taille saisie « 48 » s'affiche « 48.0 » : c'est float() qui convertit l'entier en nombre à virgule. Le champ public affiche « True », confirmant la conversion en booléen.

Partie 2 — Structures de contrôle

Cette partie transforme le programme en application interactive : un menu s'affiche en boucle et réagit au choix de l'utilisateur, jusqu'à ce qu'il décide de quitter.

NOTIONS ABORDEES boucle while · conditions if / elif / else · instruction break · indentation.

Question 5 — Créer un menu interactif

Le menu propose quatre options. La boucle while True le réaffiche indéfiniment ; le choix saisi est comparé avec if / elif / else, et l'option Quitter interrompt la boucle avec break. Un choix non prévu affiche un message d'erreur puis réaffiche le menu.

REPONSE

```
# --- Partie 2 Structure de controle ---
while True:
    print("\n=====")
    print("1. Ajouter un dataset")
    print("2. Afficher les datasets")
    print("3. Rechercher")
    print("4. Quitter")
    print("=====")

    choix = input("Votre choix : ")

    if choix == "1":
        print("Vous avez choisi : Ajouter un dataset")
    elif choix == "2":
        print("Vous avez choisi : Afficher les datasets")
    elif choix == "3":
```

```

        print("Vous avez choisi : Rechercher")
    elif choix == "4":
        print("Au revoir !")
        break
    else:
        print("Choix invalide, réessayez.")

```

Résultat de l'exécution

Le menu se réaffiche après chaque action et ne se termine que lorsque l'utilisateur saisit 4 :

```

3. Rechercher
4. Quitter
=====
Votre choix : 1
Vous avez choisi : Ajouter un dataset

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 2
Vous avez choisi : Afficher les datasets

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 3
Vous avez choisi : Rechercher

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 4
Au revoir !
PS C:\Users\NGOM PC\Desktop\datasetManager>

```

Un choix non prévu (par exemple 9) affiche « Choix invalide, réessayez. » sans arrêter le programme.

Partie 3 — Dictionnaires

Cette partie regroupe toutes les métadonnées d'un dataset dans une seule structure : un dictionnaire, où chaque information est associée à une clé (une étiquette).

NOTIONS ABORDEES dictionnaire { } · paires clé : valeur · accès par la clé dataset["clé"].

Question 6 — Créer un dictionnaire pour stocker les métadonnées

Les sept variables saisies sont regroupées dans un dictionnaire dataset. Chaque clé (« nom », « domaine »...) est associée à sa valeur. On accède ensuite à une information par sa clé, par exemple dataset["nom"].

REPONSE

```

# --- Dictionnaire du dataset ---
dataset = {
    "nom": nom,
    "domaine": domaine,
    "lignes": lignes,
    "colonnes": colonnes,

```

```

        "taille": taille,
        "format": format_dataset,
        "public": public,
    }

    print(dataset)

```

Grâce au dictionnaire, le résumé peut désormais lire directement les valeurs par leur clé, ce qui rend le code plus lisible :

```

print("\n==== Résumé du dataset =====")
print(f"Nom      : {dataset['nom']}")
print(f"Domaine   : {dataset['domaine']}")
print(f"Lignes    : {dataset['lignes']}")
print(f"Colonnes   : {dataset['colonnes']}")
print(f"Taille     : {dataset['taille']} Mo")
print(f"Format     : {dataset['format']}")
print(f"Public      : {dataset['public']}")
print("=====")

```

Résultat de l'exécution

L'affichage `print(dataset)` montre la fiche complète du dataset sous forme de paires clé : valeur :

```

PS C:\Users\NGOM PC\Desktop\datasetManager> python .\main.py
Nom du dataset : Titanic
Domaine : Transport
Nombre de lignes : 891
Nombre de colonnes : 28
Taille en Mo : 48
Format (csv ou json) : csv
Public ? (true ou false) : true
{'nom': 'Titanic', 'domaine': 'Transport', 'lignes': 891, 'colonnes': 28, 'taille': 48.0, 'format': 'csv', 'public': True}

==== Résumé du dataset ====
Nom      : Titanic
Domaine  : Transport
Lignes   : 891
Colonnes : 28
Taille   : 48.0 Mo
Format   : csv
Public   : True
=====
PS C:\Users\NGOM PC\Desktop\datasetManager>

```

Toutes les métadonnées sont désormais regroupées dans une seule structure, entre accolades.

Partie 4 — Tuples

Cette partie fige la liste des domaines autorisés dans un tuple (une collection ordonnée et non modifiable), puis vérifie que le domaine saisi en fait partie.

NOTIONS ABORDEES tuple () · collection immuable · opérateur d'appartenance in.

Question 7 — Créer un tuple des domaines autorisés

Les cinq domaines autorisés sont stockés dans un tuple, une structure adaptée à des valeurs fixes puisqu'elle ne peut être ni modifiée ni agrandie.

REPONSE

```
# --- Domaines autorisés (tuple) ---
domaines_autorises = ("Santé", "Finance", "Agriculture", "Transport", "Education")
```

| Question 8 — Vérifier que le domaine saisi appartient au tuple

Après la saisie du domaine, l'opérateur in teste s'il figure dans le tuple. Un message confirme la validité ; sinon un avertissement est affiché, sans interrompre le programme.

REPONSE

```
domaine = input("Domaine : ")

if domaine in domaines_autorises:
    print(f"Domaine « {domaine} » valide.")
else:
    print(f"Attention : « {domaine} » n'est pas dans la liste des domaines autorisés.")
```

| Résultat de l'exécution

Test avec un domaine valide, puis avec un domaine absent de la liste :

```
PS C:\Users\NGOM PC\Desktop\datasetManager> python .\main.py
Nom du dataset : Titanic
Domaine : Transport
Domaine « Transport » valide.
Nombre de lignes : 891
Nombre de colonnes : 28
Taille en Mo : 48
Format (csv ou json) : csv
Public ? (true ou false) : true
{'nom': 'Titanic', 'domaine': 'Transport', 'lignes': 891, 'colonnes': 28, 'taille': 48.0, 'format': 'csv', 'public': True}

===== Résumé du dataset =====
Nom      : Titanic
Domaine  : Transport
Lignes   : 891
Colonnes : 28
Taille   : 48.0 Mo
Format   : csv
Public   : True
=====
PS C:\Users\NGOM PC\Desktop\datasetManager> python .\main.py
Nom du dataset : Titanic 2
Domaine : test
Attention : « test » n'est pas dans la liste des domaines autorisés.
Nombre de lignes : █
```

Un domaine présent dans le tuple (Transport) est validé ; un domaine absent (Cuisine) déclenche l'avertissement sans arrêter le programme.

Partie 5 — Listes

Cette partie fait passer l'application d'un seul dataset à un catalogue : une liste stocke plusieurs datasets, et le menu permet de les ajouter, afficher, rechercher, trier, modifier et supprimer.

Question 9 — Créer la liste des datasets

Une liste vide est créée au démarrage ; chaque dataset ajouté (un dictionnaire) y est empilé. La liste est donc une liste de dictionnaires.

REPONSE

```
# --- Liste des datasets ---
datasets = []
```

Question 10 — Ajouter, afficher, rechercher, trier, modifier, supprimer

La saisie est déplacée dans l'option « Ajouter » : à chaque appui, un nouveau dictionnaire est créé puis ajouté à la liste avec append(). L'affichage parcourt la liste avec une boucle for. La recherche, la modification et la suppression retrouvent un dataset par son nom ; le tri utilise sort() avec la clé « nom ». Une pause en fin de boucle garde le résultat lisible avant de réafficher le menu.

REPONSE

```
while True:
    print("\n=====")
    print("1. Ajouter un dataset")
    print("2. Afficher les datasets")
    print("3. Rechercher un dataset")
    print("4. Trier les datasets")
    print("5. Modifier un dataset")
    print("6. Supprimer un dataset")
    print("7. Quitter")
    print("=====")

    choix = input("Votre choix : ")

    if choix == "1":
        nom = input("Nom du dataset : ")
        domaine = input("Domaine : ")
        if domaine not in domaines_autorises:
            print(f"Attention : « {domaine} » n'est pas un domaine autorisé.")
        lignes = int(input("Nombre de lignes : "))
        colonnes = int(input("Nombre de colonnes : "))
        taille = float(input("Taille en Mo : "))
        format_dataset = input("Format (csv ou json) : ")
        public = input("Public ? (true ou false) : ") == "true"

        dataset = {
            "nom": nom,
            "domaine": domaine,
            "lignes": lignes,
            "colonnes": colonnes,
            "taille": taille,
            "format": format_dataset,
            "public": public,
        }
        datasets.append(dataset)
        print(f"Dataset « {nom} » ajouté. Total : {len(datasets)}")

    elif choix == "2":
        if len(datasets) == 0:
            print("Aucun dataset enregistré.")
        else:
```



```

        for d in datasets:
            print(f"- {d['nom']} ({d['domaine']}, {d['lignes']} lignes, format {d['format']})")

    elif choix == "3":
        recherche = input("Nom du dataset à rechercher : ")
        trouve = False
        for d in datasets:
            if d["nom"] == recherche:
                print(f"Trouvé : {d['nom']} ({d['domaine']}, {d['lignes']} lignes)")
                trouve = True
        if not trouve:
            print("Aucun dataset ne porte ce nom.")

    elif choix == "4":
        datasets.sort(key=lambda d: d["nom"])
        print("Datasets triés par nom.")
        for d in datasets:
            print(f"- {d['nom']}")

    elif choix == "5":
        recherche = input("Nom du dataset à modifier : ")
        trouve = False
        for d in datasets:
            if d["nom"] == recherche:
                d["domaine"] = input("Nouveau domaine : ")
                d["lignes"] = int(input("Nouveau nombre de lignes : "))
                print(f"Dataset « {recherche} » modifié.")
                trouve = True
        if not trouve:
            print("Aucun dataset ne porte ce nom.")

    elif choix == "6":
        recherche = input("Nom du dataset à supprimer : ")
        trouve = False
        for d in datasets:
            if d["nom"] == recherche:
                datasets.remove(d)
                print(f"Dataset « {recherche} » supprimé.")
                trouve = True
                break
        if not trouve:
            print("Aucun dataset ne porte ce nom.")

    elif choix == "7":
        print("Au revoir !")
        break

    else:
        print("Choix invalide, réessayez.")

    input("\nAppuyez sur Entrée pour continuer...")

```

Résultat de l'exécution

Ajout de quatre datasets, puis test de chaque action (recherche, tri, modification, suppression) :

```
File Edit Selection View Go Run ... datasetManager Update python + - ...  
Explorer  
datasetManager  
main.py  
Problems Output Debug Console Terminal Ports  
4. Trier les datasets  
5. Modifier un dataset  
6. Supprimer un dataset  
7. Quitter  
=====  
Votre choix : Budget  
Choix invalide, réessayez.  
  
Appuyez sur Entrée pour continuer...  
  
=====  
1. Ajouter un dataset  
2. Afficher les datasets  
3. Rechercher un dataset  
4. Trier les datasets  
5. Modifier un dataset  
6. Supprimer un dataset  
7. Quitter  
=====  
Votre choix : 1  
Nom du dataset : Budget  
Domaine : Finance  
Nombre de lignes : 300  
Nombre de colonnes : 15  
Taille en Mo : 12  
Format (csv ou json) : csv  
Public ? (true ou false) : true  
Dataset « Budget » ajouté. Total : 3  
  
Appuyez sur Entrée pour continuer...  
  
File Edit Selection View Go Run ... datasetManager Update python + - ...  
Explorer  
datasetManager  
main.py  
Problems Output Debug Console Terminal Ports  
=====  
Votre choix : 1  
Nom du dataset : Covid  
Domaine : Santé  
Nombre de lignes : 5000  
Nombre de colonnes : 8  
Taille en Mo : 120  
Format (csv ou json) : json  
Public ? (true ou false) : false  
Dataset « Covid » ajouté. Total : 2  
  
Appuyez sur Entrée pour continuer...  
  
=====  
1. Ajouter un dataset  
2. Afficher les datasets  
3. Rechercher un dataset  
4. Trier les datasets  
5. Modifier un dataset  
6. Supprimer un dataset  
7. Quitter  
=====  
Votre choix : Budget  
Choix invalide, réessayez.  
  
Appuyez sur Entrée pour continuer...  
  
=====  
1. Ajouter un dataset  
2. Afficher les datasets  
3. Rechercher un dataset  
4. Trier les datasets  
5. Modifier un dataset
```

```
File Edit Selection View Go Run ... datasetManager Update
Explorer Problems Output Debug Console Terminal Ports
datasetManager
main.py

=====
Votre choix : 1
Nom du dataset : Titanic
Domaine : Transport
Nombre de lignes : 891
Nombre de colonnes : 12
Taille en Mo : 48
Format (csv ou json) : csv
Public ? (true ou false) : true
Dataset « Titanic » ajouté. Total : 1

Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Quitter
=====
Votre choix : 1
Nom du dataset : Covid
Domaine : Santé
Nombre de lignes : 5000
Nombre de colonnes : 8
Taille en Mo : 128
Format (csv ou json) : json
Public ? (true ou false) : false
Dataset « Covid » ajouté. Total : 2

Appuyez sur Entrée pour continuer...

File Edit Selection View Go Run ... datasetManager Update
Explorer Problems Output Debug Console Terminal Ports
datasetManager
main.py

PS C:\Users\NGOM PC\Desktop\datasetManager> python .\main.py
Nom du dataset : Titanic
Domaine : Transport
Nombre de lignes : 891
Nombre de colonnes : 28
Taille en Mo : 48
Format (csv ou json) : csv
Public ? (true ou false) : true
{'nom': 'Titanic', 'domaine': 'Transport', 'lignes': 891, 'colonnes': 28, 'taille': 48.0, 'format': 'csv', 'public': True}

==== Résumé du dataset ====
Nom      : Titanic
Domaine  : Transport
Lignes   : 891
Colonnes : 28
Taille   : 48.0 Mo
Format   : csv
Public   : True
=====
PS C:\Users\NGOM PC\Desktop\datasetManager>
```

```
File Edit Selection View Go Run ... datasetManager Update
Explorer Problems Output Debug Console Terminal Ports
datasetManager
main.py

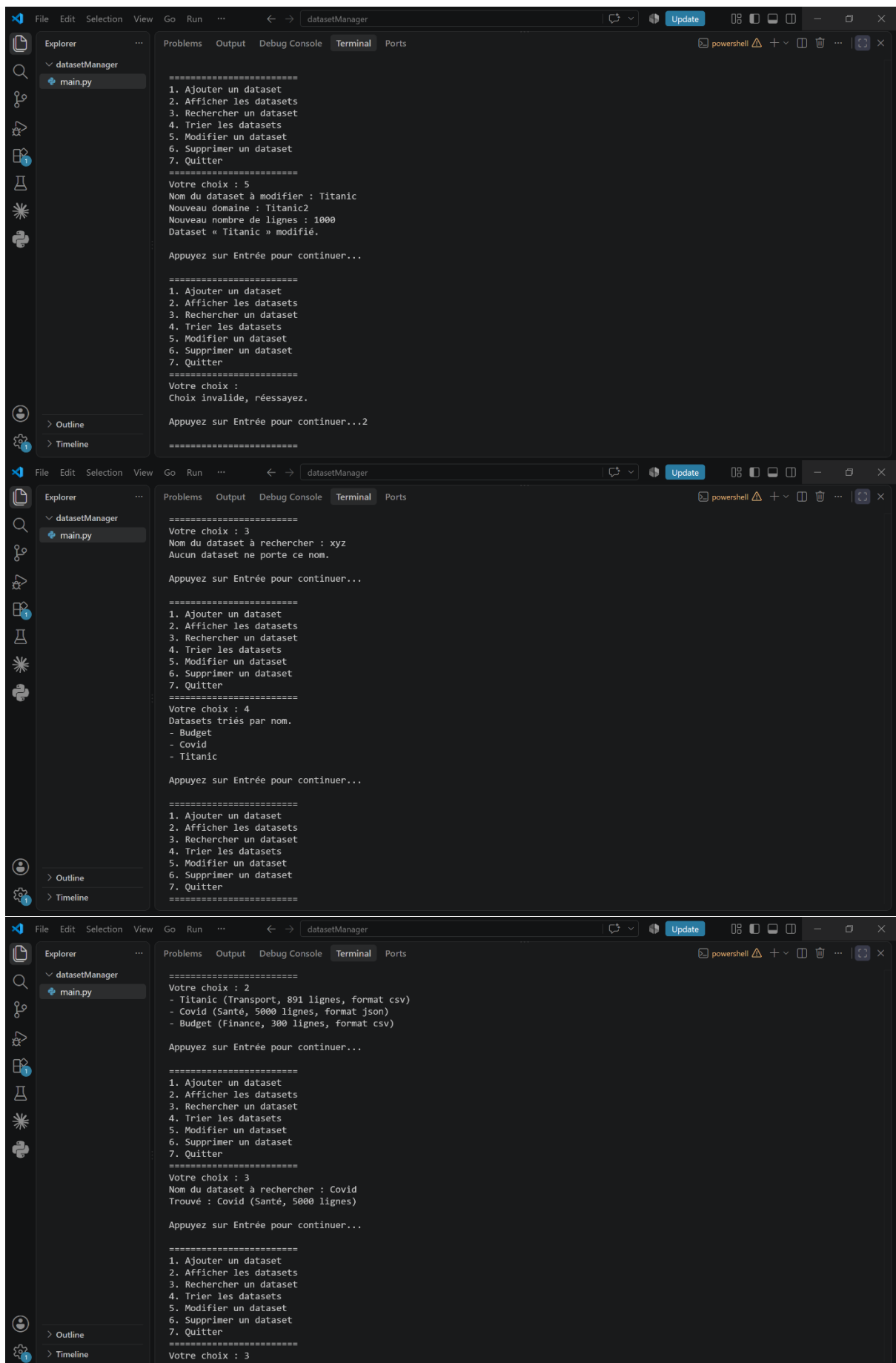
- Budget (Finance, 300 lignes, format csv)
- Covid (Santé, 5000 lignes, format json)
- Titanic (Titanic2, 1000 lignes, format csv)

Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Quitter
=====
Votre choix : 6
Nom du dataset à supprimer : Covid
Dataset « Covid » supprimé.

Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Quitter
=====
Votre choix : 7
Au revoir !
PS C:\Users\NGOM PC\Desktop\datasetManager>
```



Chaque fonctionnalité se comporte comme attendu : l'ajout empile dans la liste, l'affichage la parcourt, la recherche distingue présence et absence, le tri réordonne par nom, la modification met à jour la fiche et la suppression la retire.

Partie 6 — Compréhensions (listes et dictionnaires)

Cette partie calcule et affiche les statistiques du catalogue à l'aide des compréhensions, une écriture compacte pour parcourir la liste et compter ou filtrer en une seule ligne. Les statistiques sont proposées comme nouvelle option du menu.

NOTIONS ABORDEES compréhension de liste [x for ... if ...] · compréhension de dictionnaire {clé: valeur for ...} · fonction sum() · fonction len() · méthode .items() · formatage {valeur:.1f}.

Question 11 — Afficher les statistiques

Une option « Statistiques » est ajoutée au menu (Quitter devient l'option 8). Le total de lignes est obtenu avec sum() sur une expression génératrice ; le nombre de datasets publics, privés, CSV et JSON est obtenu en comptant (len()) des compréhensions de liste filtrées ; la répartition par domaine est construite avec une compréhension de dictionnaire, puis parcourue avec .items().

REPONSE

```
elif choix == "7":
    if len(datasets) == 0:
        print("Aucun dataset enregistré.")
    else:
        total = len(datasets)
        total_lignes = sum(d["lignes"] for d in datasets)
        moyenne_colonnes = sum(d["colonnes"] for d in datasets) / total
        publics = len([d for d in datasets if d["public"]])
        prives = total - publics
        nb_csv = len([d for d in datasets if d["format"] == "csv"])
        nb_json = len([d for d in datasets if d["format"] == "json"])

        print(f"Nombre de datasets : {total}")
        print(f"Nombre total de lignes : {total_lignes}")
        print(f"Nombre moyen de colonnes : {moyenne_colonnes:.1f}")
        print(f"Datasets publics : {publics}")
        print(f"Datasets privés : {prives}")
        print(f"Datasets CSV : {nb_csv}")
        print(f"Datasets JSON : {nb_json}")

        repartition = {dom: len([d for d in datasets if d["domaine"] == dom])
                        for dom in domaines_autorises}
        print("Répartition par domaine :")
        for dom, nb in repartition.items():
            print(f" {dom} : {nb}")
```

Résultat de l'exécution

Après avoir ajouté plusieurs datasets, l'option 7 affiche le récapitulatif statistique :

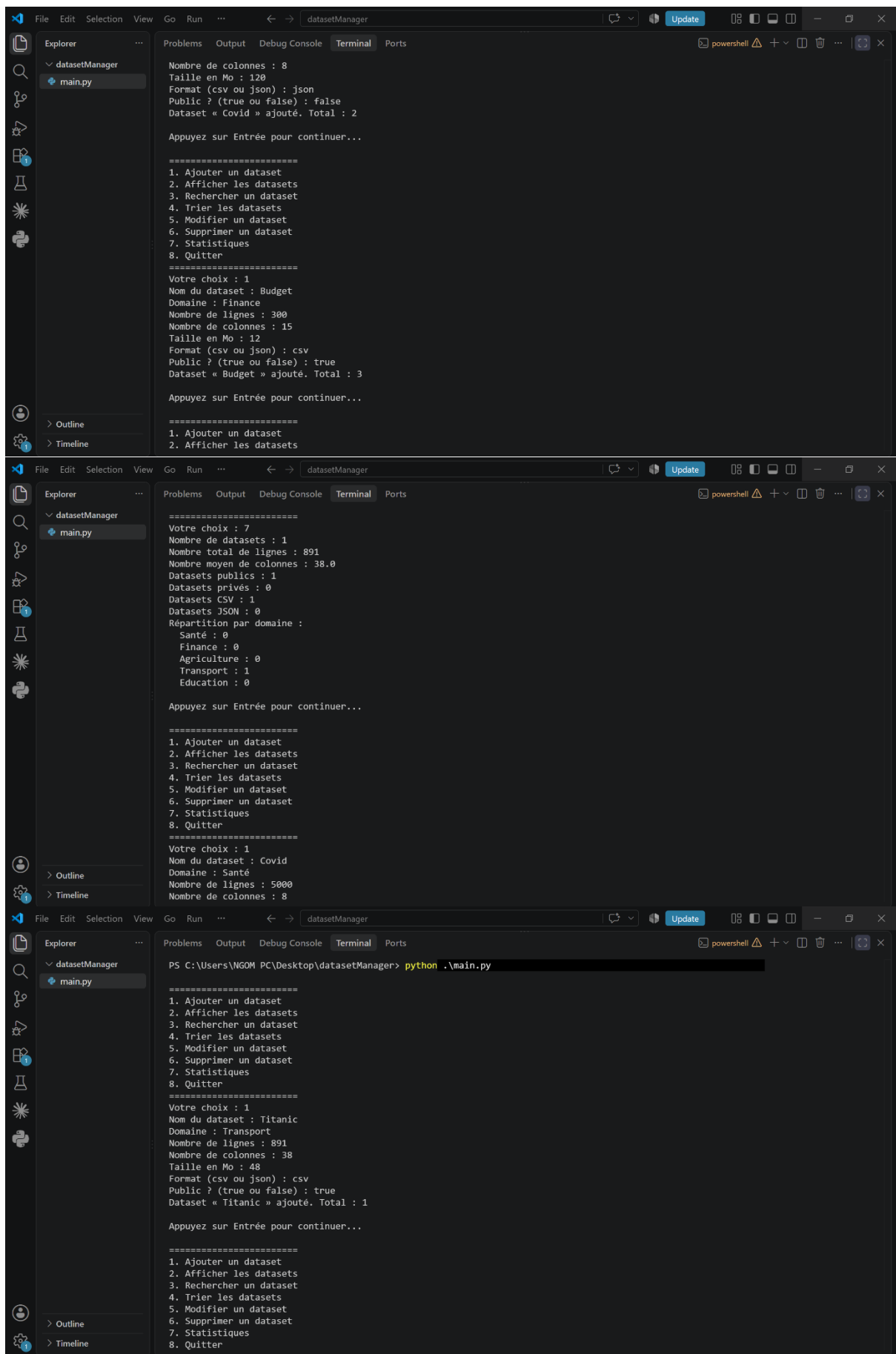
```
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Quitter
=====
Votre choix : 7
Nombre de datasets : 3
Nombre total de lignes : 6191
Nombre moyen de colonnes : 20.3
Datasets publics : 2
Datasets privés : 1
Datasets CSV : 2
Datasets JSON : 1
Répartition par domaine :
  Santé : 1
  Finance : 1
  Agriculture : 0
  Transport : 1
  Education : 0

=====
1. Ajouter un dataset
2. Afficher les datasets
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Quitter
=====
Votre choix : 8
Au revoir !
PS C:\Users\NGOM PC\Desktop\datasetManager> git add .
```

```
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Quitter
=====
Votre choix : 2
- Titanic (Transport, 891 lignes, format csv)
- Covid (Santé, 5000 lignes, format json)
- Budget (Finance, 300 lignes, format csv)

Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Quitter
=====
Votre choix : 7
Nombre de datasets : 3
Nombre total de lignes : 6191
Nombre moyen de colonnes : 20.3
Datasets publics : 2
Datasets privés : 1
Datasets CSV : 2
Datasets JSON : 1
```



Les compteurs (publics/privés, CSV/JSON) et la répartition par domaine sont calculés directement à partir de la liste, sans boucle explicite grâce aux compréhensions.

Partie 7 — Fichiers

Jusqu'ici, les datasets ne vivaient qu'en mémoire et disparaissaient à la fermeture. Cette partie rend les données persistantes : elles sont écrites dans un fichier `datasets.csv`, puis relues au besoin. Deux options sont ajoutées au menu (Quitter devient l'option 10).

NOTIONS ABORDEES module `csv` · fonction `open()` avec `with` · modes « `w` » (écriture) et « `r` » (lecture) · `csv.DictWriter` / `csv.DictReader` · `writeheader()` / `writerow()` · reconversion des types à la lecture.

Question 12 — Sauvegarder et recharger les données

Le module `csv` est importé en début de fichier. Il fournit un écrivain et un lecteur adaptés aux dictionnaires, ce qui correspond exactement à la structure de nos datasets.

```
import csv
```

SAUVEGARDE (OPTION 8)

Le fichier est ouvert en écriture ; un `DictWriter` écrit d'abord la ligne d'en-tête (les noms de colonnes), puis une ligne par dataset.

```
elif choix == "8":
    with open("datasets.csv", "w", newline="", encoding="utf-8") as fichier:
        colonnes_csv = ["nom", "domaine", "lignes", "colonnes",
                        "taille", "format", "public"]
        writer = csv.DictWriter(fichier, fieldnames=colonnes_csv)
        writer.writeheader()
        for d in datasets:
            writer.writerow(d)
    print(f"{len(datasets)} dataset(s) sauvegardé(s) dans datasets.csv")
```

RECHARGEMENT (OPTION 9)

La liste est vidée pour éviter les doublons, le fichier est lu avec un `DictReader` (chaque ligne redevient un dictionnaire), et chaque champ est reconverti dans son type d'origine : un fichier CSV ne contient que du texte.

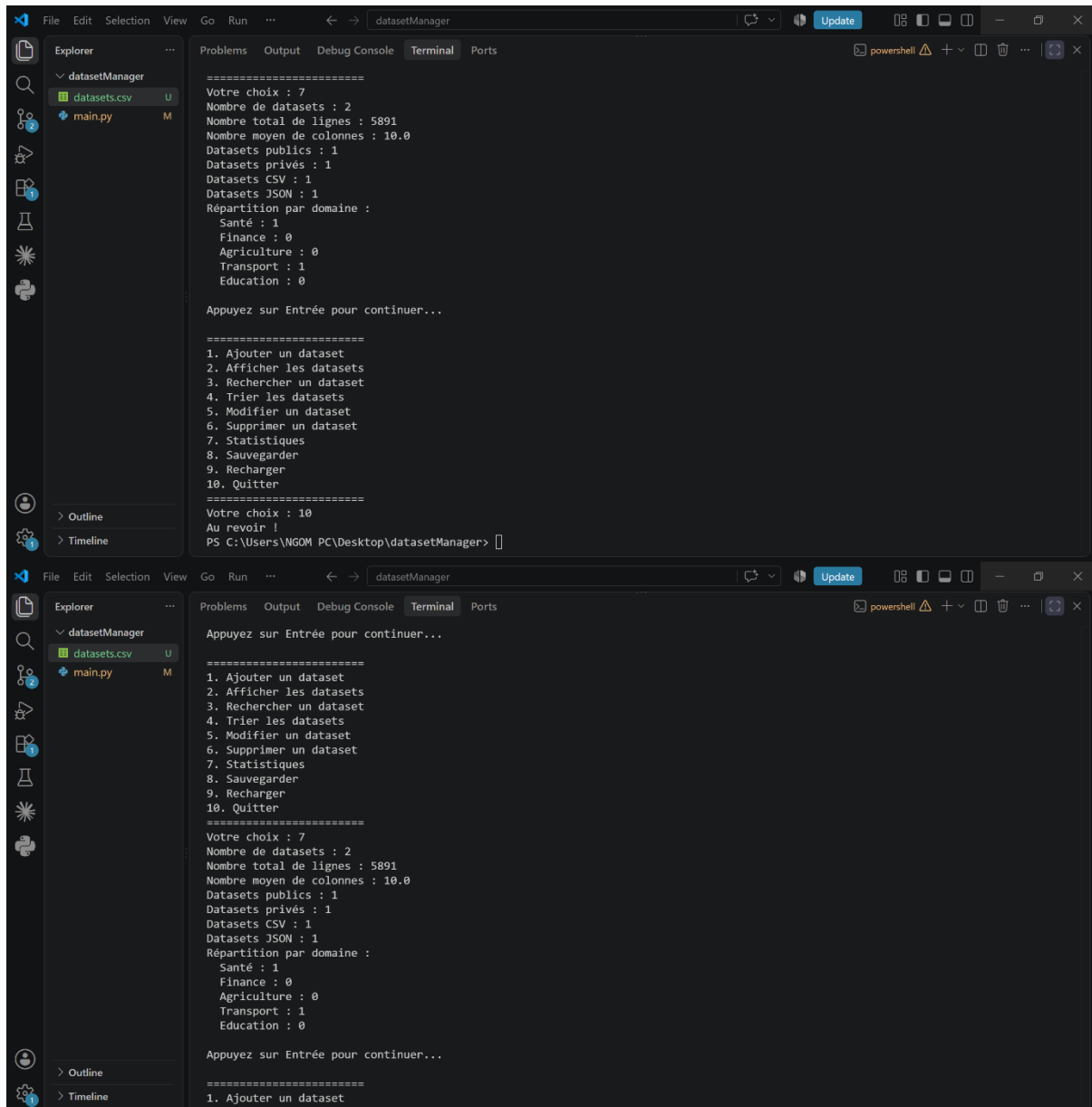
```
elif choix == "9":
    datasets.clear()
    with open("datasets.csv", "r", newline="", encoding="utf-8") as fichier:
        reader = csv.DictReader(fichier)
        for ligne in reader:
            ligne["lignes"] = int(ligne["lignes"])
            ligne["colonnes"] = int(ligne["colonnes"])
            ligne["taille"] = float(ligne["taille"])
            ligne["public"] = ligne["public"] == "True"
```



```
datasets.append(ligne)
print(f"{len(datasets)} dataset(s) rechargé(s) depuis datasets.csv")
```

Résultat de l'exécution

Ajout de datasets, sauvegarde (option 8), puis fermeture, relance et rechargement (option 9) : les données sont bien retrouvées.



```
=====
Votre choix : 7
Nombre de datasets : 2
Nombre total de lignes : 5891
Nombre moyen de colonnes : 10.0
Datasets publics : 1
Datasets privés : 1
Datasets CSV : 1
Datasets JSON : 1
Répartition par domaine :
  Santé : 1
  Finance : 0
  Agriculture : 0
  Transport : 1
  Education : 0

Appuyez sur Entrée pour continuer...

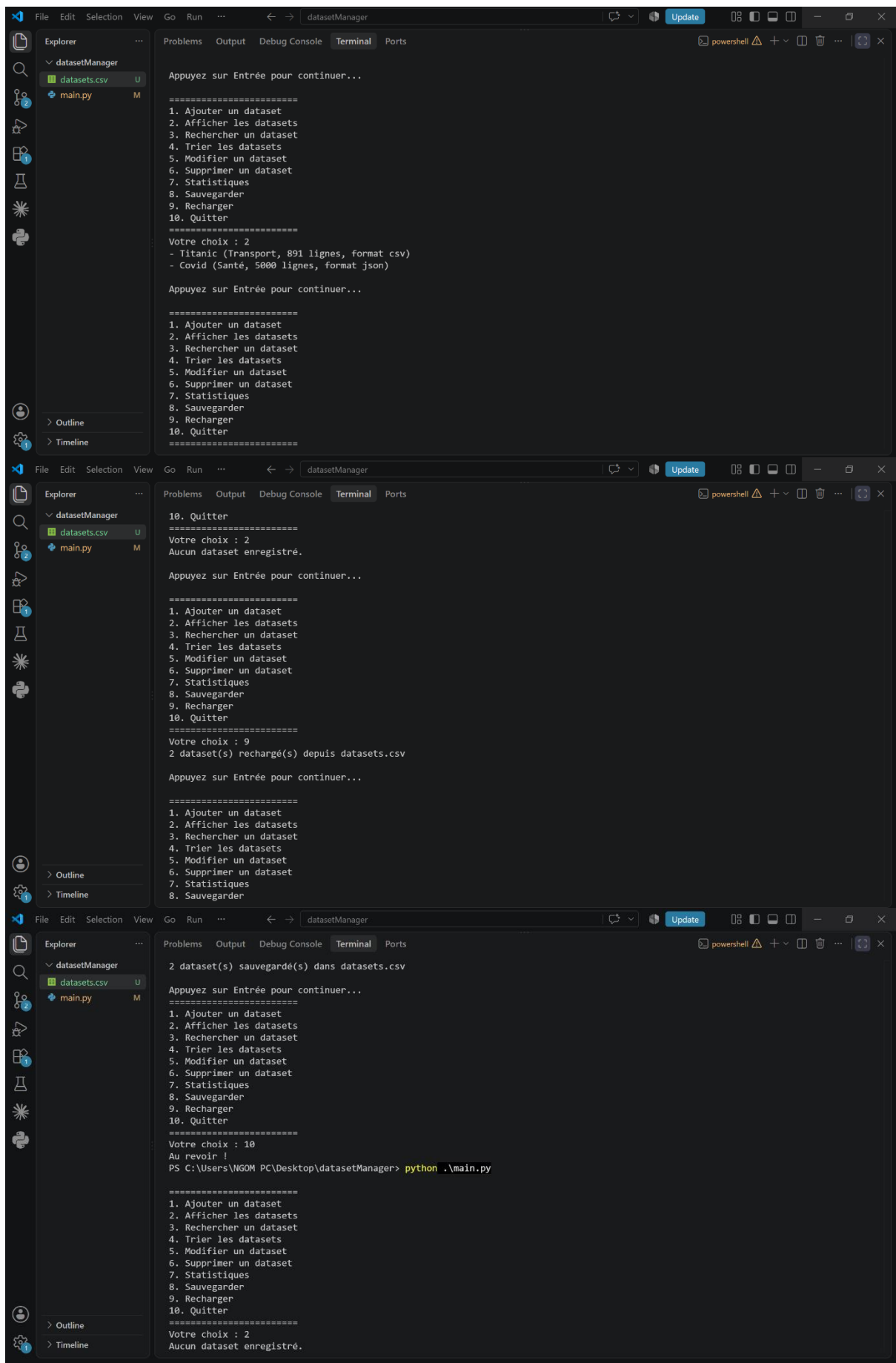
=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 10
Au revoir !
PS C:\Users\NGOM PC\Desktop\datasetManager>

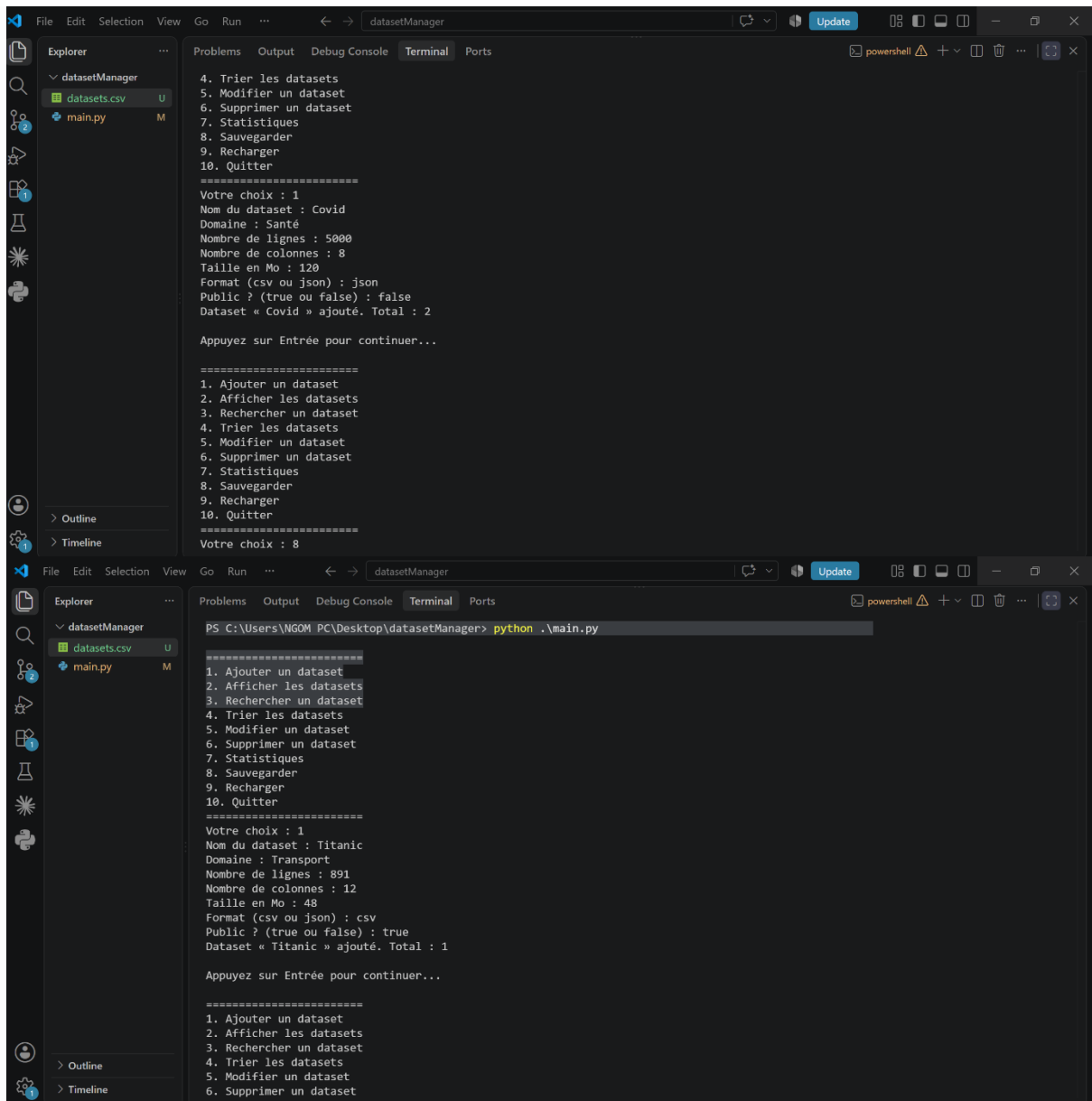
=====
Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 7
Nombre de datasets : 2
Nombre total de lignes : 5891
Nombre moyen de colonnes : 10.0
Datasets publics : 1
Datasets privés : 1
Datasets CSV : 1
Datasets JSON : 1
Répartition par domaine :
  Santé : 1
  Finance : 0
  Agriculture : 0
  Transport : 1
  Education : 0

Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
```





Le fichier datasets.csv généré contient une ligne d'en-tête puis une ligne par dataset :

```
nom,domaine,lignes,colonnes,taille,format,public
Titanic,Transport,891,12,48.0,csv,True
Covid,Santé,5000,8,120.0,json,False
```

Les datasets survivent désormais à la fermeture du programme, et les statistiques restent correctes après rechargement — preuve que les types sont bien restaurés.

Partie 8 — Exceptions

Une exception est une erreur qui interrompt brutalement le programme. Cette partie protège les points sensibles avec le bloc try / except, pour que le programme affiche un message et continue au lieu de planter.

NOTIONS ABORDEES exception · bloc try / except · exceptions ValueError et FileNotFoundError · continuité du programme après une erreur.

Question 13 — Gérer les erreurs constatées

Quatre erreurs sont traitées : un texte saisi à la place d'un nombre, un fichier inexistant, un fichier vide, et un dataset recherché absent.

TEXTE AU LIEU D'UN NOMBRE (AJOUT ET MODIFICATION)

La saisie des valeurs numériques est placée dans un try ; si int() ou float() échoue, l'exception ValueError est rattrapée et un message s'affiche, sans interrompre le programme.

```
if choix == "1":
    nom = input("Nom du dataset : ")
    domaine = input("Domaine : ")
    if domaine not in domaines_autorises:
        print(f"Attention : « {domaine} » n'est pas un domaine autorisé.")
    try:
        lignes = int(input("Nombre de lignes : "))
        colonnes = int(input("Nombre de colonnes : "))
        taille = float(input("Taille en Mo : "))
        format_dataset = input("Format (csv ou json) : ")
        public = input("Public ? (true ou false) : ") == "true"
        dataset = {"nom": nom, "domaine": domaine, "lignes": lignes,
                  "colonnes": colonnes, "taille": taille,
                  "format": format_dataset, "public": public}
        datasets.append(dataset)
        print(f"Dataset « {nom} » ajouté. Total : {len(datasets)}")
    except ValueError:
        print("Erreur : lignes, colonnes et taille doivent être des nombres. Dataset non ajouté.")
```

La modification (option 5) est protégée de la même façon autour de la nouvelle valeur du nombre de lignes.

FICHIER INEXISTANT OU VIDE (RECHARGEMENT)

L'ouverture du fichier est placée dans un try : si datasets.csv n'existe pas, FileNotFoundError est rattrapée ; s'il existe mais ne contient aucune donnée, un message « fichier vide » est affiché. La liste n'est vidée qu'après une ouverture réussie.

```
elif choix == "9":
    try:
        with open("datasets.csv", "r", newline="", encoding="utf-8") as fichier:
            reader = csv.DictReader(fichier)
            datasets.clear()
            for ligne in reader:
                ligne["lignes"] = int(ligne["lignes"])
                ligne["colonnes"] = int(ligne["colonnes"])
                ligne["taille"] = float(ligne["taille"])
                ligne["public"] = ligne["public"] == "True"
                datasets.append(ligne)
            if len(datasets) == 0:
                print("Le fichier datasets.csv est vide.")
            else:
                print(f"{len(datasets)} dataset(s) rechargé(s) depuis datasets.csv")
```

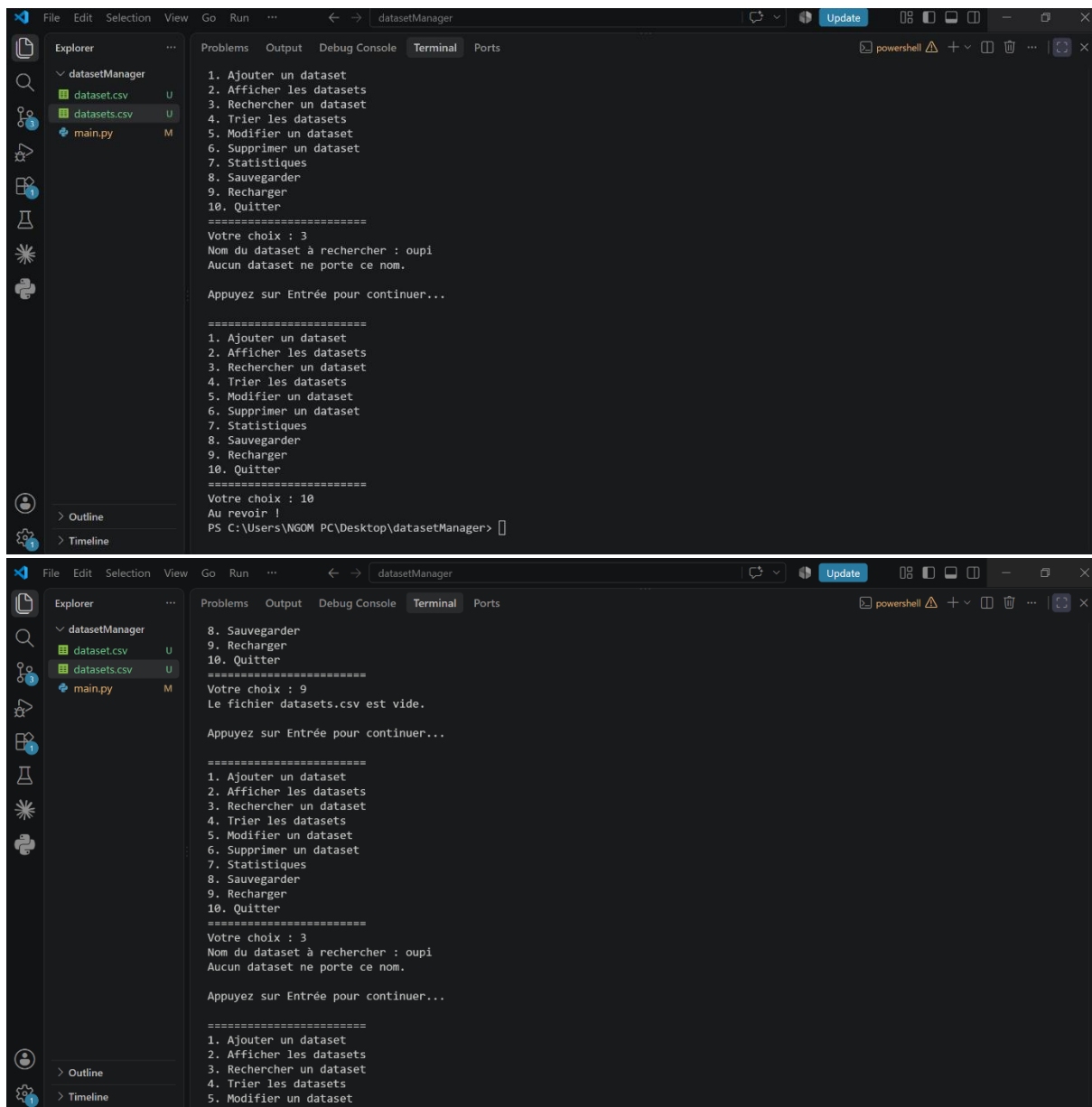
```
except FileNotFoundError:
    print("Le fichier datasets.csv n'existe pas encore. Faites d'abord une
sauvegarde.")
```

DATASET RECHERCHE INEXISTANT

Ce cas n'est pas une exception mais une vérification logique, déjà en place dans la recherche : un drapeau booléen trouve reste à False si aucun nom ne correspond, et le message « Aucun dataset ne porte ce nom » est alors affiché.

Résultat de l'exécution

Test des quatre cas : nombre invalide, fichier absent, fichier vide et recherche introuvable. Le programme réagit par un message et poursuit dans tous les cas.



```
File Edit Selection View Go Run ... datasetManager
Explorer
  datasetManager
    dataset.csv U
    datasets.csv U
    main.py M
Problems Output Debug Console Terminal Ports
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 3
Nom du dataset à rechercher : oupi
Aucun dataset ne porte ce nom.

Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 10
Au revoir !
PS C:\Users\NGOM PC\Desktop\datasetManager>

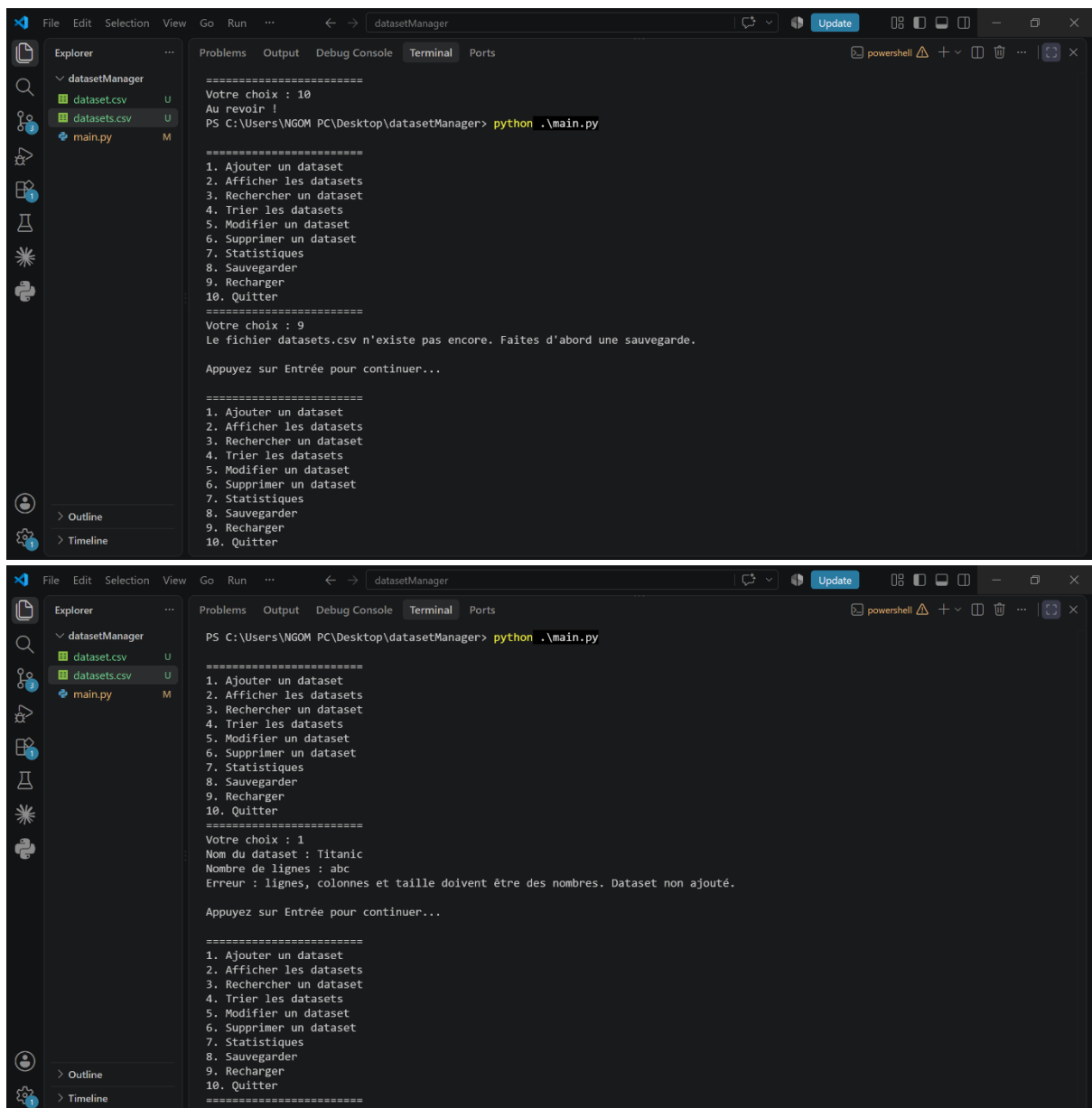
File Edit Selection View Go Run ... datasetManager
Explorer
  datasetManager
    dataset.csv U
    datasets.csv U
    main.py M
Problems Output Debug Console Terminal Ports
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 9
Le fichier datasets.csv est vide.

Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 3
Nom du dataset à rechercher : oupi
Aucun dataset ne porte ce nom.

Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
```



Aucune de ces erreurs n'interrompt le programme : le menu réapparaît systématiquement après le message.

Partie 9 — Fonctions

Cette partie ne change pas le comportement du programme : elle le réorganise. Chaque action, jusque-là écrite dans la boucle du menu, est extraite dans une fonction nommée. Le programme principal devient court et lisible : il se contente d'appeler la bonne fonction selon le choix.

NOTIONS ABORDEES définition avec `def` · appel de fonction `nom()` · liste `datasets` partagée entre les fonctions · instruction `return` · programme principal réduit à des appels.

Question 14 — Refactoriser le programme en fonctions

La liste datasets et le tuple des domaines sont déclarés une seule fois en haut du fichier ; toutes les fonctions travaillent sur cette même liste. Les fonctions sont définies avant la boucle principale.

Exemple avec l'ajout :

EXEMPLE DE FONCTION

```
def ajouter_dataset():
    nom = input("Nom du dataset : ")
    domaine = input("Domaine : ")
    if domaine not in domaines_autorises:
        print(f"Attention : « {domaine} » n'est pas un domaine autorisé.")
    try:
        lignes = int(input("Nombre de lignes : "))
        colonnes = int(input("Nombre de colonnes : "))
        taille = float(input("Taille en Mo : "))
        format_dataset = input("Format (csv ou json) : ")
        public = input("Public ? (true ou false) : ") == "true"
        dataset = {"nom": nom, "domaine": domaine, "lignes": lignes,
                  "colonnes": colonnes, "taille": taille,
                  "format": format_dataset, "public": public}
        datasets.append(dataset)
        print(f"Dataset « {nom} » ajouté. Total : {len(datasets)}")
    except ValueError:
        print("Erreur : lignes, colonnes et taille doivent être des nombres. Dataset non ajouté.")
```

Les autres actions sont extraites de la même façon, ce qui donne les dix fonctions demandées : afficher_menu, ajouter_dataset, afficher_datasets, rechercher_dataset, trier_dataset, modifier_dataset, supprimer_dataset, statistiques, sauvegarder et recharger. Leurs corps reprennent le code des parties précédentes.

LE PROGRAMME PRINCIPAL

La boucle affiche le menu et appelle la fonction correspondant au choix saisi :

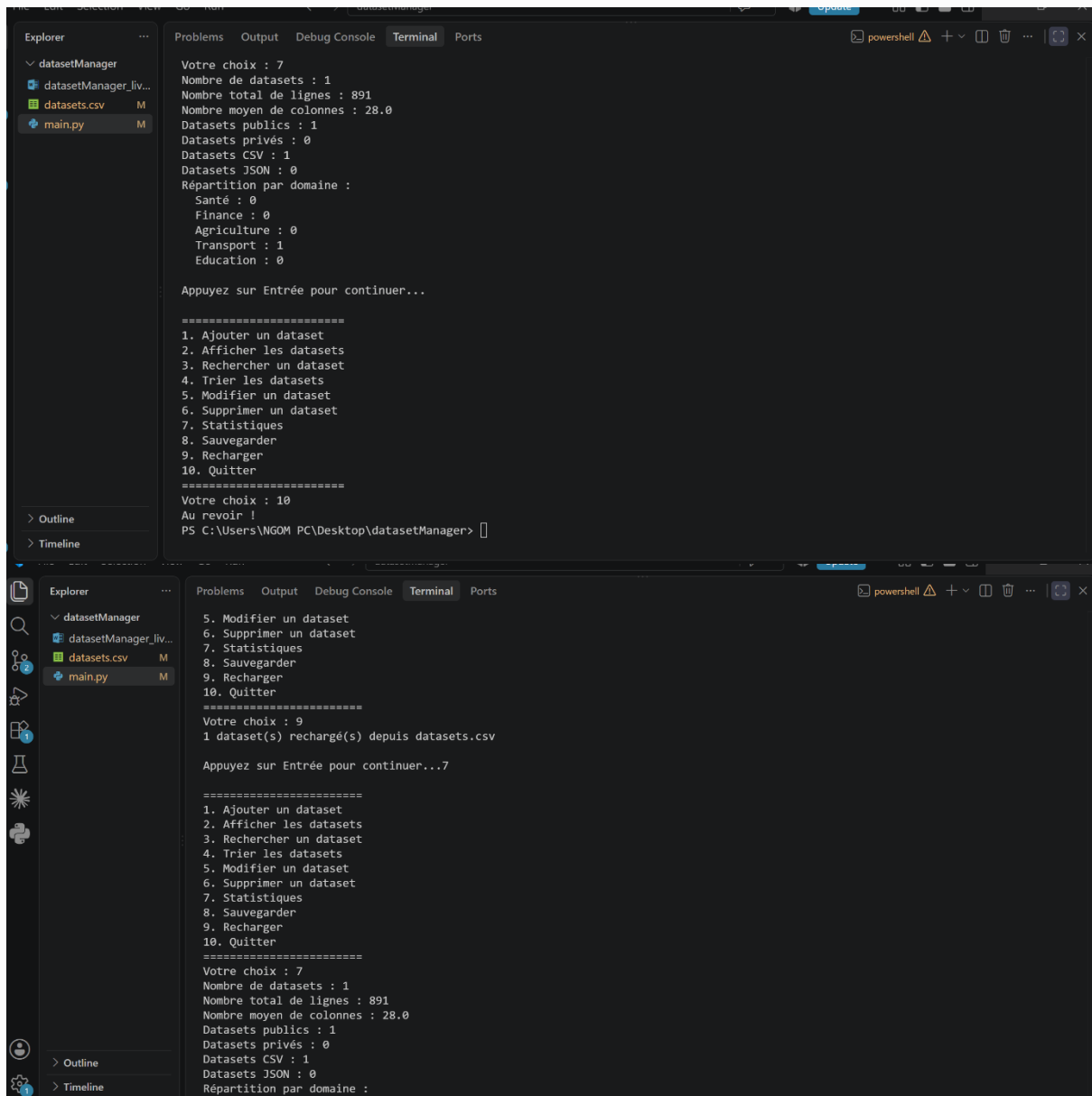
```
while True:
    afficher_menu()
    choix = input("Votre choix : ")

    if choix == "1":
        ajouter_dataset()
    elif choix == "2":
        afficher_datasets()
    elif choix == "3":
        rechercher_dataset()
    elif choix == "4":
        trier_dataset()
    elif choix == "5":
        modifier_dataset()
    elif choix == "6":
        supprimer_dataset()
    elif choix == "7":
        statistiques()
    elif choix == "8":
        sauvegarder()
    elif choix == "9":
        recharger()
    elif choix == "10":
        print("Au revoir !")
        break
    else:
        print("Choix invalide, réessayez.")
```

```
input("\nAppuyez sur Entrée pour continuer...")
```

Résultat de l'exécution

Le comportement reste identique à celui des parties précédentes ; seule l'organisation du code a changé :



```
Votre choix : 7
Nombre de datasets : 1
Nombre total de lignes : 891
Nombre moyen de colonnes : 28.0
Datasets publics : 1
Datasets privés : 0
Datasets CSV : 1
Datasets JSON : 0
Répartition par domaine :
Santé : 0
Finance : 0
Agriculture : 0
Transport : 1
Education : 0

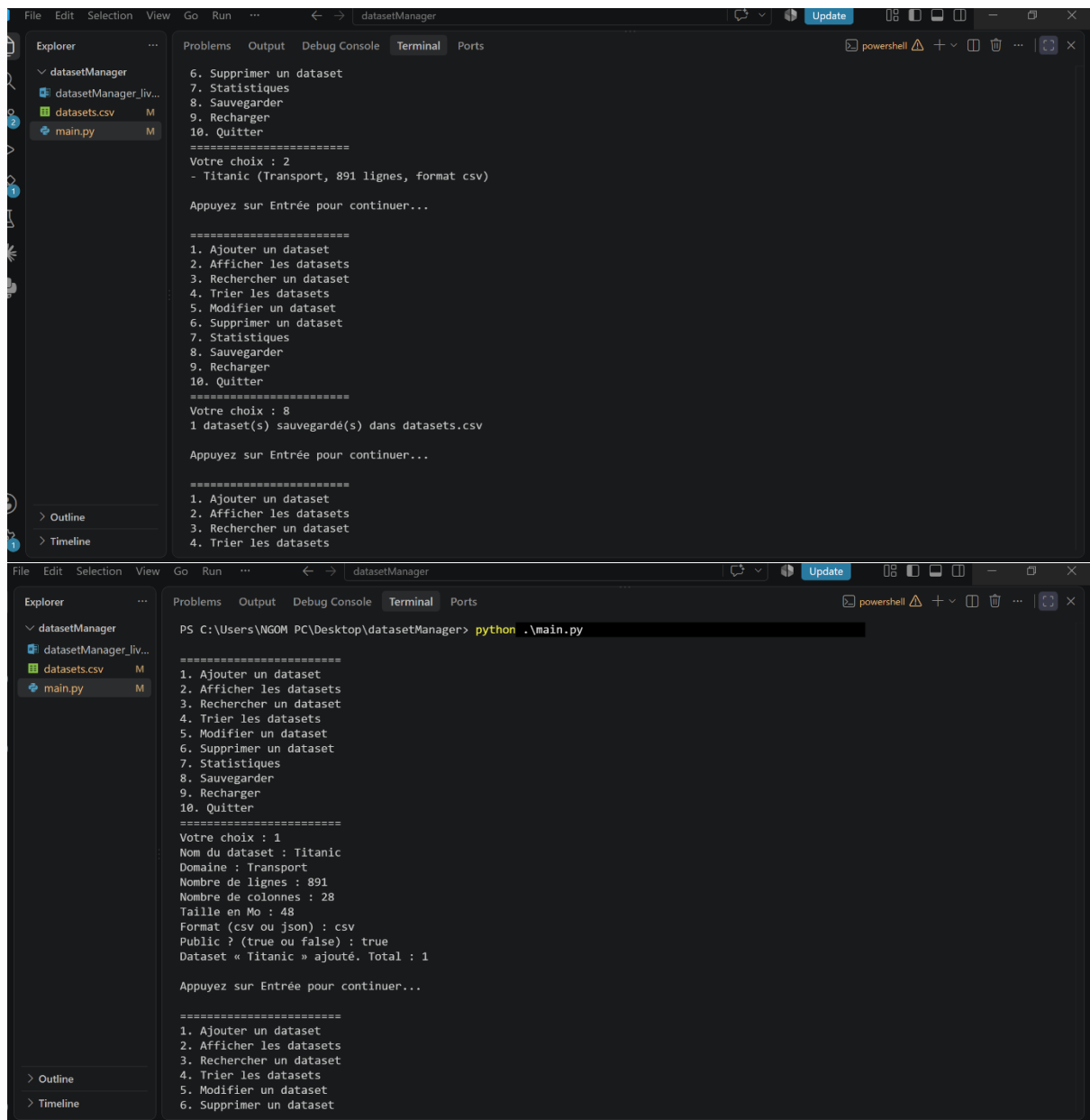
Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 10
Au revoir !
PS C:\Users\NGOM PC\Desktop\datasetManager>

=====
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 9
1 dataset(s) rechargé(s) depuis datasets.csv

Appuyez sur Entrée pour continuer...7

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 7
Nombre de datasets : 1
Nombre total de lignes : 891
Nombre moyen de colonnes : 28.0
Datasets publics : 1
Datasets privés : 0
Datasets CSV : 1
Datasets JSON : 0
Répartition par domaine :
```

Le programme fonctionne comme avant, mais le code est désormais découpé en fonctions claires et réutilisables.

Partie 10 — Modules

Jusqu'ici tout le code tenait dans un seul fichier. Cette partie le répartit en plusieurs modules (des fichiers `.py`), chacun avec une responsabilité, pour faciliter la maintenance. Le comportement du programme reste identique.

NOTIONS ABORDEES module (fichier .py) · import et from ... import ... · passage de datasets en paramètre · point d'entrée main.py.

Question 15 — Répartir le code dans plusieurs modules

Le code est réparti en quatre fichiers : menu.py (affichage du menu), gestion.py (ajout, affichage, recherche, tri, modification, suppression, sauvegarde, rechargement), statistiques.py (calcul des statistiques) et main.py (le point d'entrée). Comme les fonctions ne sont plus dans le même fichier que la liste, celle-ci leur est transmise en paramètre.

ORGANISATION DES FICHIERS

```
datasetManager/  
├── main.py          # point d'entrée  
├── menu.py          # afficher_menu  
├── gestion.py       # gestion des datasets  
├── statistiques.py  # statistiques  
└── datasets.csv     # données sauvegardées
```

gestion.py regroupe les fonctions qui manipulent la liste, chacune recevant datasets en paramètre : ajouter_dataset(datasets, domaines_autorises), afficher_datasets(datasets), rechercher_dataset(datasets), trier_dataset(datasets), modifier_dataset(datasets), supprimer_dataset(datasets), sauvegarder(datasets) et recharger(datasets). statistiques.py contient statistiques(datasets, domaines_autorises), et menu.py contient afficher_menu(). Les corps de ces fonctions sont ceux de la Partie 9.

MAIN.PY — LE POINT D'ENTREE

main.py importe les fonctions depuis les trois modules, crée la liste, puis lance la boucle qui appelle chaque fonction en lui passant datasets :

```
from menu import afficher_menu  
from gestion import (  
    ajouter_dataset, afficher_datasets, rechercher_dataset,  
    trier_dataset, modifier_dataset, supprimer_dataset,  
    sauvegarder, recharger,  
)  
from statistiques import statistiques  
  
domaines_autorises = ("Santé", "Finance", "Agriculture", "Transport", "Education")  
datasets = []  
  
while True:  
    afficher_menu()  
    choix = input("Votre choix : ")  
  
    if choix == "1":  
        ajouter_dataset(datasets, domaines_autorises)  
    elif choix == "2":  
        afficher_datasets(datasets)  
    elif choix == "3":  
        rechercher_dataset(datasets)  
    elif choix == "4":  
        trier_dataset(datasets)  
    elif choix == "5":  
        modifier_dataset(datasets)  
    elif choix == "6":  
        supprimer_dataset(datasets)  
    elif choix == "7":  
        statistiques(datasets, domaines_autorises)
```

```

elif choix == "8":
    sauvegarder(datasets)
elif choix == "9":
    recharger(datasets)
elif choix == "10":
    print("Au revoir !")
    break
else:
    print("Choix invalide, réessayez.")

input("\nAppuyez sur Entrée pour continuer...")

```

Résultat de l'exécution

Le programme est lancé avec `python main.py` depuis le dossier ; le comportement est identique à la Partie 9, mais le code est réparti en modules :

```

Votre choix : 7
Nombre de datasets : 1
Nombre total de lignes : 891
Nombre moyen de colonnes : 28.0
Datasets publics : 1
Datasets privés : 0
Datasets CSV : 1
Datasets JSON : 0
Répartition par domaine :
Santé : 0
Finance : 0
Agriculture : 0
Transport : 1
Education : 0

Appuyez sur Entrée pour continuer...

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 10
Au revoir !
PS C:\Users\NGOM PC\Desktop\datasetManager>

```

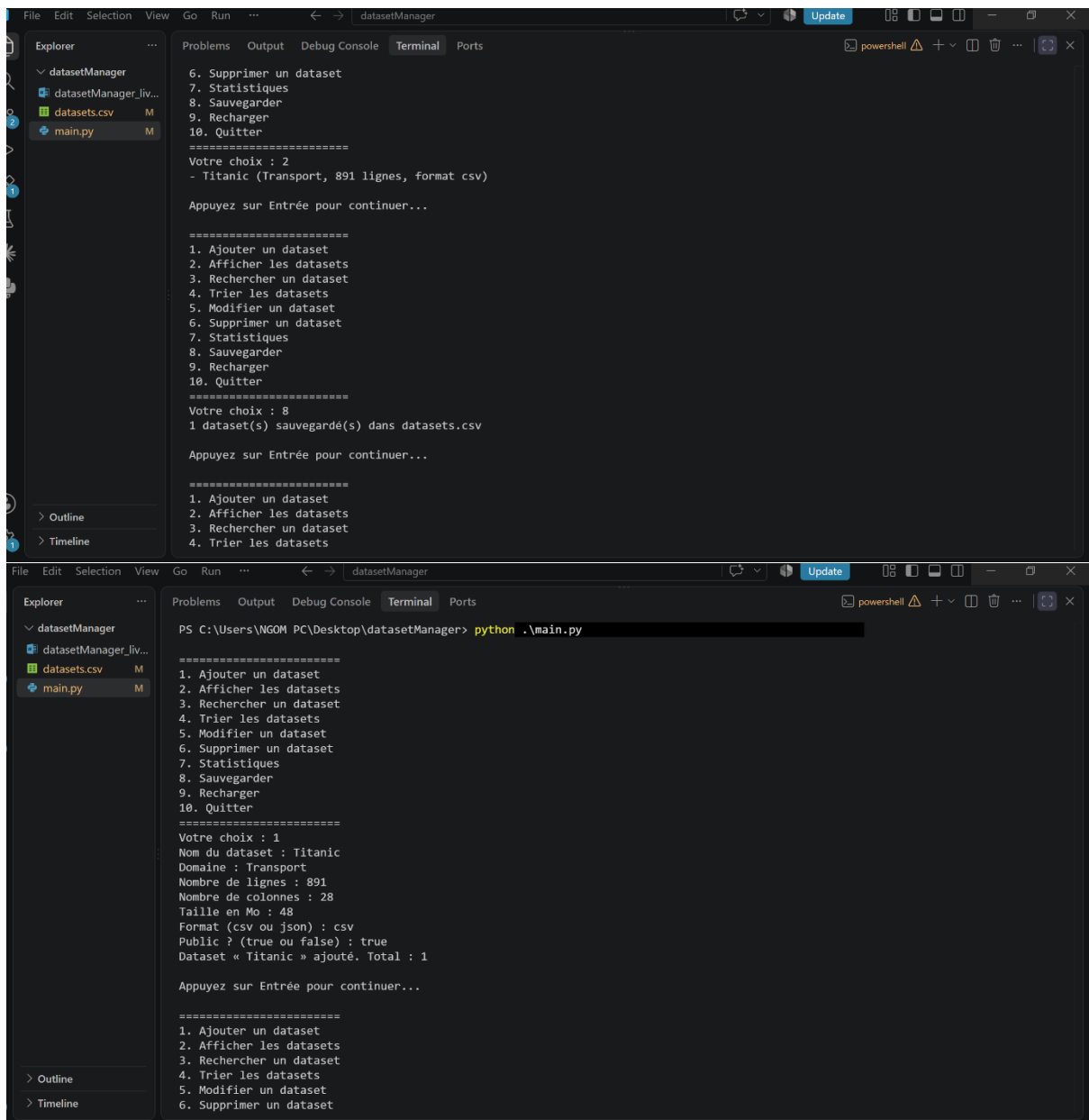
```

5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 9
1 dataset(s) rechargé(s) depuis datasets.csv

Appuyez sur Entrée pour continuer...7

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Trier les datasets
5. Modifier un dataset
6. Supprimer un dataset
7. Statistiques
8. Sauvegarder
9. Recharger
10. Quitter
=====
Votre choix : 7
Nombre de datasets : 1
Nombre total de lignes : 891
Nombre moyen de colonnes : 28.0
Datasets publics : 1
Datasets privés : 0
Datasets CSV : 1
Datasets JSON : 0
Répartition par domaine :

```



La séparation en modules ne change rien pour l'utilisateur ; elle rend simplement le code plus organisé et plus facile à maintenir.

Partie 11 — Packages

Cette partie organise les modules dans des packages, c'est-à-dire des dossiers contenant les fichiers `.py` et un fichier `__init__.py`. Le code est réparti par grand thème, selon l'arborescence demandée par le sujet.

NOTIONS ABORDEES package (dossier + `__init__.py`) · imports pointés `package.module` · dossier `data/` pour les fichiers de données.

Question 16 — Découper l'application en packages

Les modules sont regroupés dans trois packages — `interface` (menu, affichage), `datasets` (gestion, statistiques) et `stockage` (`csv_manager`, `json_manager`) — et un dossier `data/` pour les données. Chaque package contient un fichier `__init__.py` (vide) qui le rend importable ; `data/` n'est pas un package et n'en contient donc pas.

ARBORESCENCE DU PROJET

```
datasetManager/  
├── main.py  
├── interface/  
│   ├── __init__.py  
│   ├── menu.py  
│   └── affichage.py  
├── datasets/  
│   ├── __init__.py  
│   ├── gestion.py  
│   └── statistiques.py  
├── stockage/  
│   ├── __init__.py  
│   ├── csv_manager.py  
│   └── json_manager.py  
└── data/  
    ├── datasets.csv  
    └── datasets.json
```

Les fichiers de données étant dans `data/`, les chemins d'accès deviennent « `data/datasets.csv` » et « `data/datasets.json` » ; le programme doit être lancé depuis le dossier racine `datasetManager`.

MAIN.PY — IMPORTS PAR PACKAGES

`main.py` importe les fonctions via des chemins pointés qui suivent l'arborescence (`package.module`) :

```
from interface.menu import afficher_menu  
from interface.affichage import afficher_datasets  
from datasets.gestion import (  
    ajouter_dataset, rechercher_dataset, trier_dataset,  
    modifier_dataset, supprimer_dataset,  
)  
from datasets.statistiques import statistiques  
from stockage.csv_manager import sauvegarder_csv, recharger_csv  
from stockage.json_manager import sauvegarder_json, recharger_json
```

La boucle principale reste celle de la Partie 10. Lancé depuis la racine, le programme fonctionne comme auparavant, mais son code est désormais clairement organisé par responsabilité au sein des packages.